Leave One Out: The Jackknife and Cross-Validation

Mark M. Fredrickson (mfredric@umich.edu)

Module 3: Resample

Computational Methods in Statistics and Data Science (DATASCI 406)

Leaving out one observation

Deck 1 resampled with replacement. This deck recomputes a statistic n times, leaving out one observation each time.

- **Jackknife**: an SE from the spread of the n values.
- **Bootstrap-t**: an interval that uses a jackknife SE inside each bootstrap sample.
- **Cross-validation**: prediction error, predicting each observation from the other $n - 1$.

The Jackknife

Estimating the variance of a statistic

Suppose we need the standard error for T.

- In some cases, we know how to estimate (e.g., $s/\sqrt{n}$ for $\bar{X}$)
- For most statistics there is no formula. The bootstrap estimates it, but with Monte Carlo error and can be computationally expensive.
- Instead, we could use the n **subsets** that leave out one observation.

Goal: combine the n leave-one-out values of T into an estimate of $\text{Var}(T)$ that gives s^2/n when $T = \bar{x}$.

Leave-one-out values

The ith **leave-one-out value** is T computed without x_i,

$$T_{(i)} = T(x_1, \ldots, x_{i-1}, x_{i+1}, \ldots, x_n), \quad i = 1, \ldots, n.$$

In R, `x[-i]` drops x_i.

```
mdeq <- read.csv("../../data/flint_mdeq.csv")
x <- mdeq$lead
n <- length(x)
t_loo <- sapply(1:n, function(i) mean(x[-i]))
range(t_loo)
```

```
## [1] 5.929 7.414
```

Leave-one-out means

Leaving out x_i removes it from the sum,

$$T_{(i)} = \frac{n\bar{x} - x_i}{n-1}.$$

Averaging over i,

$$\bar{T}_{(\cdot)} = \frac{1}{n}\sum_{i=1}^{n} \frac{n\bar{x} - x_i}{n-1} = \frac{n^2\bar{x} - n\bar{x}}{n(n-1)} = \bar{x},$$

so

$$T_{(i)} - \bar{T}_{(\cdot)} = \frac{n\bar{x} - x_i - (n-1)\bar{x}}{n-1} = \frac{\bar{x} - x_i}{n-1}.$$

Matching s^2/n

A first try: the sample variance of the $T_{(i)}$. For the mean,

$$\frac{1}{n-1}\sum_{i=1}^{n}\left(T_{(i)}-\bar{T}_{(\cdot)}\right)^2=\frac{1}{(n-1)^3}\sum_{i=1}^{n}(x_i-\bar{x})^2=\frac{s^2}{(n-1)^2}.$$

This is too small by a factor of $(n-1)^2/n$. Correcting,

$$\frac{(n-1)^2}{n}\cdot\frac{1}{n-1}\sum_{i=1}^{n}\left(T_{(i)}-\bar{T}_{(\cdot)}\right)^2=\frac{n-1}{n}\sum_{i=1}^{n}\left(T_{(i)}-\bar{T}_{(\cdot)}\right)^2=\frac{s^2}{n}.$$

The jackknife SE

The same formula for any T is the **jackknife standard error**,

$$\widehat{\mathrm{SE}}_{\mathrm{jack}}(T) = \sqrt{\frac{n-1}{n}\sum_{i=1}^{n}\left(T_{(i)} - \bar{T}_{(\cdot)}\right)^2}, \qquad \bar{T}_{(\cdot)} = \frac{1}{n}\sum_{i=1}^{n} T_{(i)}.$$

- For $\bar{x}$ it is exactly $s/\sqrt{n}$. The bootstrap SE only approaches $\hat{\sigma}/\sqrt{n}$.
- For smooth functions of means, such as a variance, a correlation or a ratio of means, it is a good approximation. It fails for quantiles (below).

In R

```
jack_se <- function(t_loo) {
  n <- length(t_loo)
  sqrt((n - 1) / n * sum((t_loo - mean(t_loo))^2))
}
c(jackknife = jack_se(t_loo), formula = sd(x) / sqrt(n))
```

```
## jackknife   formula
##     1.703     1.703
```

jack_se takes the leave-one-out values, so it works for any statistic.

The Flint correlation

Deck 1 bootstrapped the correlation of `lx` and `ly`, the logged first-draw and flushed samples from 271 homes.

```
r_loo <- sapply(1:n_vt, function(i) cor(lx[-i], ly[-i]))
boot_cor <- replicate(B, {
  i <- sample(n_vt, replace = TRUE)
  cor(lx[i], ly[i])
})
c(r = cor(lx, ly), jackknife = jack_se(r_loo), bootstrap = sd(boot_cor))
```

```
##         r jackknife bootstrap
##   0.64760   0.04370   0.04319
```

Jackknife or bootstrap?

- Jackknife: n evaluations of T, no random numbers.
- Bootstrap: B evaluations, with Monte Carlo error. The jackknife is cheaper when $n < B$.
- Only the bootstrap gives a whole distribution, and so intervals.
- The jackknife needs a smoother T than the bootstrap does.

When the Jackknife Fails

The median of the MDEQ samples

```
med_loo <- sapply(1:n, function(i) median(x[-i]))
table(med_loo)
```

```
## med_loo
##  3
## 71
```

```
boot_med <- replicate(B, median(sample(x, replace = TRUE)))
c(jackknife = jack_se(med_loo), bootstrap = sd(boot_med))
```

```
## jackknife bootstrap
##    0.0000    0.6784
```

11 of the 71 samples equal the median, 3 ppb, so every leave-one-out median is 3 and the jackknife SE is 0.

Leave-one-out medians take two or three values

Let $n = 2m$, so the median is $(x_{(m)} + x_{(m+1)})/2$.

- Leave out an observation at or below $x_{(m)}$: the median becomes $x_{(m+1)}$.
- Leave out one at or above $x_{(m+1)}$: it becomes $x_{(m)}$.

So the jackknife SE is

$$\frac{\sqrt{n-1}}{2}\left(x_{(m+1)} - x_{(m)}\right),$$

which depends only on the gap between the two middle observations. For odd n there are three values.

Simulation: normal data

The SE of the median of n draws from $N(0,1)$ is about $\sqrt{\pi/(2n)}$.

```
ratio <- sapply(c(20, 200, 2000), function(m) {
  replicate(200, {
    z <- rnorm(m)
    jack_se(sapply(1:m, function(i) median(z[-i]))) / sqrt(pi / (2 * m))
  })
})
colnames(ratio) <- c("n = 20", "n = 200", "n = 2000")
apply(ratio, 2, quantile, c(0.1, 0.5, 0.9))
```

```
##      n = 20 n = 200 n = 2000
## 10% 0.1205  0.1101   0.1110
## 50% 0.8311  0.6516   0.7167
## 90% 2.2867  2.3161   2.4564
```

What the jackknife handles

- **Works:** smooth functions of means, such as the mean (exactly), a variance, a correlation, a ratio of means, or a regression coefficient. For the Flint correlation the jackknife SE was 0.044, against a bootstrap SE of 0.043.
- **Fails:** sample quantiles, including the median and the 90th percentile the bootstrap handled roughly in deck 1. The MDEQ leave-one-out 90th percentiles take 3 values.
- **The difference:** $\bar{x}_{(i)}$ moves in proportion to how far x_i is from $\bar{x}$. A leave-one-out median depends only on which side of the median x_i falls.
- For quantiles, bootstrap instead, or leave out d observations at a time with $\sqrt{n} \ll d \ll n$.

The Bootstrap-t Interval

Studentizing

For normal data, $(\bar{X} - \mu)/(s/\sqrt{n}) \sim t_{n-1}$ whatever μ and σ are.

The **bootstrap-t** approximates the distribution of $W = (T - \theta)/\widehat{\mathrm{SE}}$ with

$$W_b^* = \frac{T_b^* - T}{\widehat{\mathrm{SE}}_b^*}, \quad b = 1, \ldots, B,$$

where $\widehat{\mathrm{SE}}_b^*$ comes **from bootstrap sample** b. As with the basic interval, but using W^*,

$$\left[T - \widehat{\mathrm{SE}}\, W_{1-\alpha/2}^*,\ T - \widehat{\mathrm{SE}}\, W_{\alpha/2}^* \right],$$

with $\widehat{\mathrm{SE}}$ from the original sample.

An SE inside every bootstrap sample

Each bootstrap sample needs its own $\widehat{\mathrm{SE}}_b^*$:

- **A formula**, such as $s^*/\sqrt{n}$. Most statistics have none.
- **A nested bootstrap**: $B_2 \approx 50$ to 200 resamples of each bootstrap sample, $B \times B_2$ evaluations.
- **A jackknife** in each bootstrap sample: $B \times n$ evaluations, no Monte Carlo error.

For Flint, $B \times n$ is 1,084,000 correlations. With a shortcut for all n leave-one-out values, the jackknife costs little more than T.

In R

```
r <- cor(lx, ly)
se_r <- jack_se(r_loo)
boot_w <- replicate(B, {
  i <- sample(n_vt, replace = TRUE)
  a <- lx[i]
  b <- ly[i]
  se_star <- jack_se(sapply(1:n_vt, function(j) cor(a[-j], b[-j])))
  (cor(a, b) - r) / se_star
})
quantile(boot_w, c(0.025, 0.975))
```

```
##   2.5%  97.5%
## -1.819  2.297
```

Unlike t quantiles, these are not symmetric about 0.

Bootstrap-t and percentile intervals

```
rbind(percentile = quantile(boot_cor, c(0.025, 0.975)),
      bootstrap_t = r - se_r * quantile(boot_w, c(0.975, 0.025)))
```

```
##                2.5%  97.5%
## percentile  0.5575 0.7270
## bootstrap_t 0.5472 0.7271
```

The SE of a correlation shrinks as the correlation grows. Bootstrap samples with $r^* > r$ have small SEs and large W^*, which pushes the lower end of the interval down.

Cross-Validation

Prediction

Fit a predictor $\hat{f}$ to pairs $(X_i, Y_i) \overset{\text{iid}}{\sim} F$, and predict a new Y_0 with $\hat{f}(X_0)$.

A **loss function** $L(y, \hat{y})$ scores a prediction, such as $(y - \hat{y})^2$. The **prediction error** is

$$\text{Err} = \text{E}\left(L(Y_0, \hat{f}(X_0))\right),$$

over both the sample and the new pair.

In Flint, X is the log first draw and Y the log after 2 minutes of flushing.

Training error is optimistic

Training error, $\frac{1}{n}\sum_i L(Y_i, \hat{f}(X_i))$, scores $\hat{f}$ on the points that chose it.

Simplest case: no X, $\hat{f} = \bar{Y}$, squared error.

- Expected training error: $\frac{n-1}{n}\sigma^2$.
- Expected error on a new Y_0: $\sigma^2 + \frac{\sigma^2}{n} = \frac{n+1}{n}\sigma^2$.

With more parameters, $\hat{f}$ follows the training points more closely and the gap grows.

Leave the point out

$\hat{f}_{(i)}$, fit without (X_i, Y_i), is independent of that point, so it predicts it as a new point.

Leave-one-out cross-validation estimates prediction error by

$$\mathrm{CV} = \frac{1}{n} \sum_{i=1}^{n} L\left(Y_i, \hat{f}_{(i)}(X_i)\right).$$

The loop is the jackknife's, but each $\hat{f}_{(i)}$ is compared with the point it left out.

Check: the mean

For $\hat{f} = \bar{Y}$, $Y_i - \bar{Y}_{(i)} = \frac{n}{n-1}(Y_i - \bar{Y})$, so

$$\text{CV} = \frac{1}{n} \cdot \frac{n^2}{(n-1)^2} \sum_{i=1}^{n} (Y_i - \bar{Y})^2 = \frac{n}{n-1} s^2.$$

Its expectation, $\sigma^2 + \sigma^2/(n-1)$, is exactly the prediction error of a mean of $n-1$ observations.

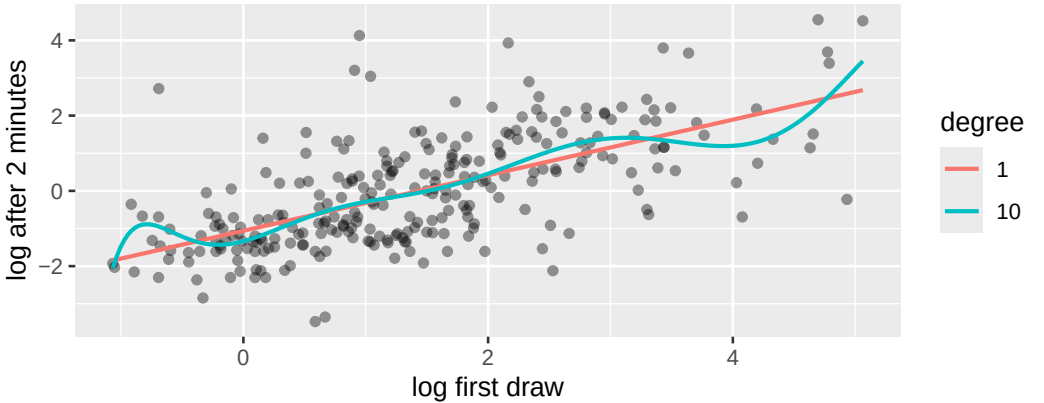
In R

```
loo_err <- ly - sapply(1:n_vt, function(i) mean(ly[-i]))
c(training = mean((ly - mean(ly))^2),
  cv = mean(loo_err^2),
  formula = n_vt / (n_vt - 1) * var(ly))
```

```
## training       cv  formula
##    2.181    2.197    2.197
```

Choosing a Polynomial Degree

Flushed against first draw



Degree 10 has a smaller training error than a line. Does it predict a new home better?

Leave-one-out for a polynomial

```
flint <- data.frame(lx, ly)
loo_mse <- function(d) {
  err <- sapply(1:n_vt, function(i) {
    fit <- lm(ly ~ poly(lx, d), data = flint[-i, ])
    flint$ly[i] - predict(fit, flint[i, ])
  })
  mean(err^2)
}
loo_mse(1)
```

```
## [1] 1.288
```

Each degree takes 271 fits.

A shortcut for least squares

For least squares, the leave-one-out residual is

$$Y_i - \hat{f}_{(i)}(X_i) = \frac{e_i}{1 - h_{ii}},$$

with e_i the residual and h_{ii} the leverage, the ith diagonal entry of the hat matrix. One fit gives all n.

```
fit <- lm(ly ~ poly(lx, 1), data = flint)
mean((resid(fit) / (1 - hatvalues(fit)))^2)
```

```
## [1] 1.288
```

This matches `loo_mse(1)`. Most predictors have no such shortcut.

k-fold cross-validation

Split the data at random into K **folds**. For each fold k:

1. fit $\hat{f}$ to the other folds,
2. predict fold k.

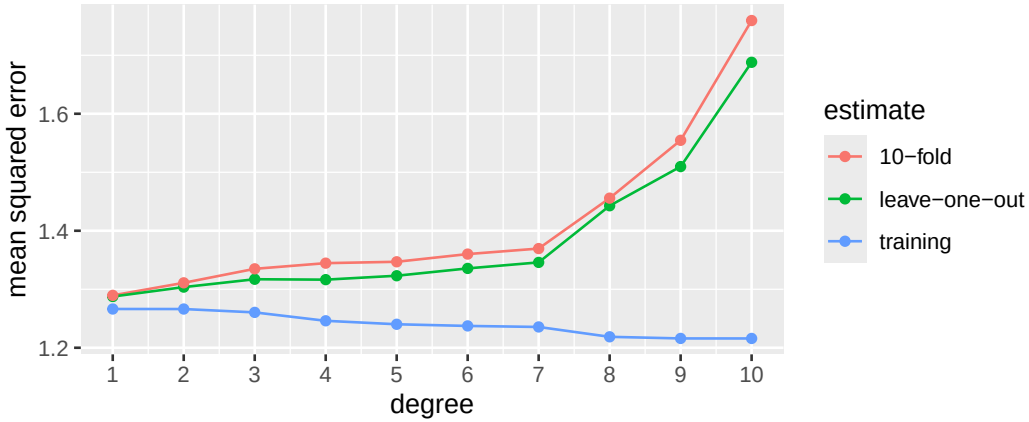
- K fits instead of n. $K = 5$ or 10 is common, and leave-one-out is $K = n$.
- Each fit uses $(K-1)/K$ of the data, so the estimate is for a slightly smaller sample.
- Random folds add Monte Carlo error. Use the same folds for every model.

In R

```
fold <- sample(rep(1:10, length.out = n_vt))
kfold_mse <- function(d) {
  err <- numeric(n_vt)
  for (k in 1:10) {
    test <- fold == k
    fit <- lm(ly ~ poly(lx, d), data = flint[!test, ])
    err[test] <- flint$ly[test] - predict(fit, flint[test, ])
  }
  mean(err^2)
}
kfold_mse(1)
```

```
## [1] 1.29
```

Training and cross-validation error by degree



Choosing the degree

```
sapply(errs[-1], which.min)
```

```
##      training leave-one-out       10-fold
##            10             1            1
```

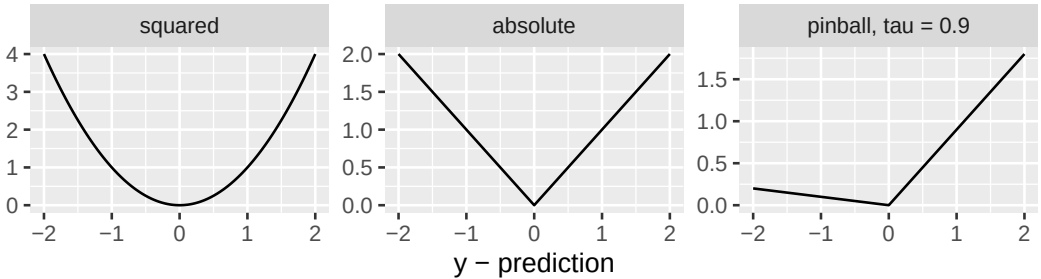
- Training error falls with every term.
- Both cross-validation estimates are smallest for a line, and climb past degree 7, where the polynomial chases a few extreme homes.
- The minimum CV error is slightly optimistic for the chosen model, since the degree was chosen to minimize it.

Choosing a Loss Function

Three loss functions

With $e = y - \hat{y}$, the **squared**, **absolute** and **pinball** losses are

$$e^2, \qquad |e|, \qquad L_\tau(e) = \begin{cases} \tau\, e & e > 0, \\ (1-\tau)(-e) & e \leq 0. \end{cases}$$



With $\tau = 0.9$, predicting 1 ppb too low costs 9 times as much as 1 ppb too high.

Each loss has its own best prediction

For a constant prediction c:

- squared loss is minimized by the **mean**,
- absolute loss by the **median**,
- pinball loss by the τ **quantile**. The derivative, $-\tau P(Y > c) + (1 - \tau)P(Y < c)$, is zero where $P(Y < c) = \tau$.

For a utility predicting lead after flushing, too low a prediction is the worse mistake. It tells a family its water is safer than it is.

Flint: leave-one-out under each loss

```
y <- vt$flush_2min
pinball <- function(e, tau = 0.9) ifelse(e > 0, tau * e, (tau - 1) * e)
preds <- list(mean = mean, median = median,
              p90 = function(z) quantile(z, 0.9, names = FALSE))
err <- sapply(preds, function(f) y - sapply(1:n_vt, function(i) f(y[-i])))
rbind(squared = colMeans(err^2), absolute = colMeans(abs(err)),
      pinball = colMeans(pinball(err)))
```

```
##              mean  median     p90
## squared  111.471 118.727 123.571
## absolute   4.439   3.330   6.960
## pinball    2.219   2.799   2.129
```

Each loss ranks first the predictor that minimizes it.

Picking a loss

- Write down what each kind of error costs, and use that loss.
- Otherwise use the field's convention, as with $\alpha = 0.05$.
- Fit and evaluate with the same loss when possible: least squares for squared error, a conditional quantile for the pinball loss.
- Cross-validation works with any loss.

Summary

Summary: the jackknife

- The jackknife SE is the spread of the $T_{(i)}$, scaled by $(n-1)/n$, the factor that makes it exactly $s/\sqrt{n}$ for $\bar{x}$.
- It works for smooth functions of means, such as a correlation. It fails for quantiles, whose leave-one-out values take two or three values.
- The bootstrap-t studentizes each T_b^* with a jackknife SE from its own bootstrap sample, at a cost of $B \times n$ evaluations.

Summary: cross-validation

- Training error underestimates prediction error.
- Leave-one-out cross-validation predicts each point from the other $n - 1$. K-fold needs only K fits.
- Choose the model with the smallest cross-validation error, using the same folds for each.
- The loss decides the best prediction: the mean, the median, or a quantile.